Presentation 4

| 2025.11.19

LEGAL AID CHICAGO

Stat 390 Simple EDA Team 2: Sabrina, Vivienne, Caroline, Hailee, Nikole

Classifying Outcomes by Final Activity

Approach:

1) Use call flow charts to verify if activity corresponds to planned final step of a call
   a) If yes, check if there are any conditions that need to be met for a successful call (e.g. spoke to an agent, final step has a meaningful duration) and use to determine rule for classifying calls
   b) Otherwise, check verbiage to see if there is a reason for a call to end at this point (e.g. Pre-Tenant Menu directs callers with an eviction case filed against them to other resources) and determine classification based on this information
2) If activity names not included in the flow charts, (i.e. internal steps of call system), find examples of calls with this activity name in their journey, looking at cases where it is and is not the final activity to assess how a successful call journey should look. From this, determine different rules and classifications of outcomes

Additional notes and code in the appendix

Classifying Outcomes by Final Activity

Questions & Issues

- GetLoggedIn<*>Agents
  - Unsure how to classify
  - Tentatively understood as the phone system checking whether there are agents logged in to answers calls for the specified queue, but there is an unknown issue and the call never progresses; calls mainly end by caller leaving
  - Is this case of abandonment or system failure?
- TransferToSafeHaven
  - Based on verbiage, unsure what selection on Family Menu corresponds to this activity
  - Seems to be transferred to chatbot agent
- LegalServerScreenPop & ScreenPopProcessComplete
  - Unsure if there is a more rigorous way to determine if the call successfully connected to a client besides looking for an agent name and time difference after the activity
- No activity
  - Is this a specific activity name? Unable to find from data
  - How was this determined?

Verifying Outcomes by Final Activity

Approach:

1) Performing a consistency check to ensure that I am seeing the same number of calls as in Krish's analysis
2) Take a general look at call journeys with specific last activities
    a) Some are more straightforward and clear cut (ex: MainMenu)
    b) Some are more ambiguous (ex: ClinicVoicemailTransfer)
3) For more ambiguous activities, look at more call journeys within there and verify that all of the possible scenarios have been captured
    a) ClinicVoicemailTransfer, DisconnectContact1 had additional scenarios identified
4) All updates are in the spreadsheet

Outcome Tagging Approach

- Summarize entire CAR dataset by Contact Session ID, grabbing the last value for each field:
  - Activity Name
  - EP Name
  - Flow Name
  - Agent Name
  - + Time difference between last activity and final row in ID
- Use case-when logic to tag outcomes using spreadsheet
  - Meeting conditions for last activity (and other fields when indicated in comments column)
  - Sub-type conditions also taken from spreadsheet, rows without “type” column filled out coded as unknown

If EP name = 'Closed Queue Menu Telephony EP', then Negative, and type = "Closed queue", otherwise Positive, type = 'Reached voicemail', if EP name = 'Closed Hours-Holidays Menu Telephony EP' then neutral, and type = 'After hours call

Pseudocode

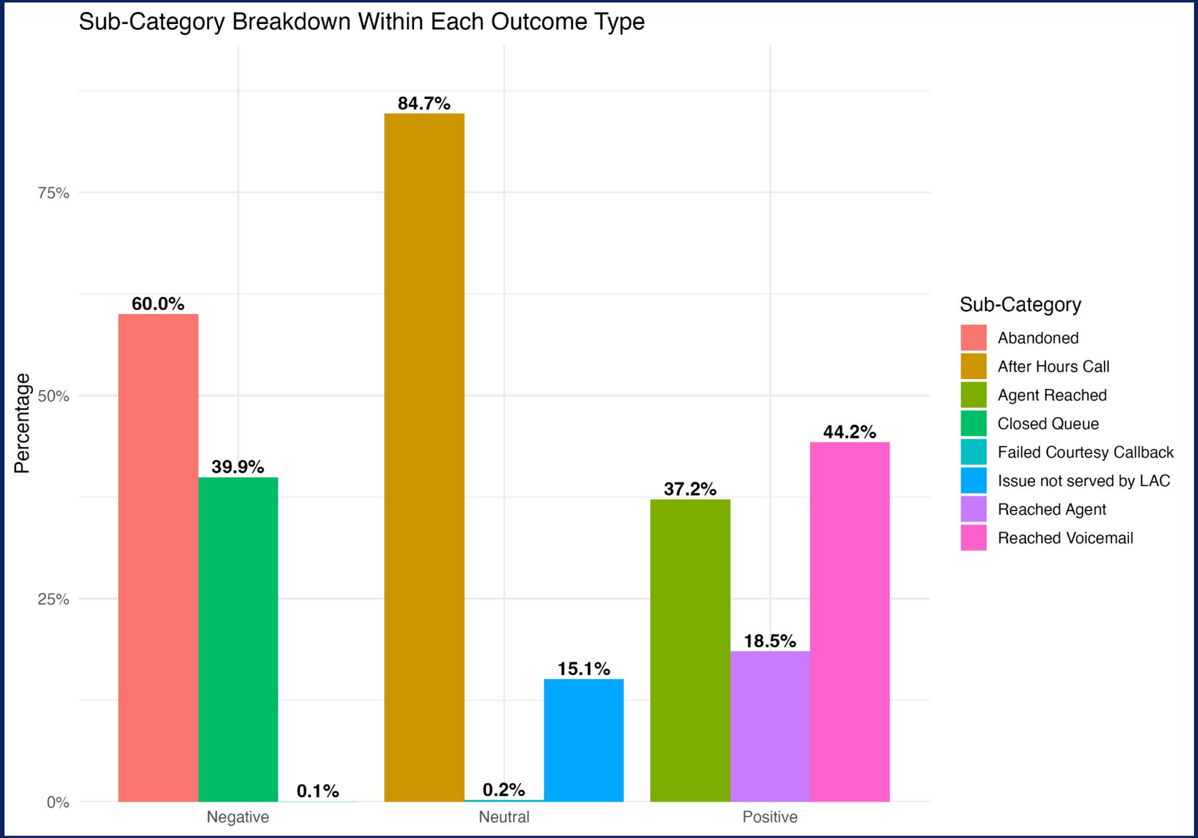
When
- Last activity = ClinicVoicemailTransfer AND EP name = “Closed Queue Menu Telephony EP’
  - THEN Negative, Closed Queue
- Last activity = ClinicVoicemailTransfer AND EP name != “Closed Queue Menu Telephony EP’
  - THEN Positive, Voicemail Reached
- Last activity = ClinicVoicemailTransfer AND EP name = ‘Closed Hours-Holiday Menu Telephony EP’
  - THEN Neutral, After Hours Call

Metric 1: Proportion of positive, negative, and neutral outcomes in the data, and their split into the above sub-categories

Main Takeaways:

Negative Outcomes:

- The majority of negative outcomes come from Abandoned calls (60%), meaning callers drop off before reaching help.
- Closed Queue is the second-largest driver (≈40%), suggesting many callers reach the system when services are unavailable.
- Very few negative calls end due to Failed Courtesy Callbacks (<1%).

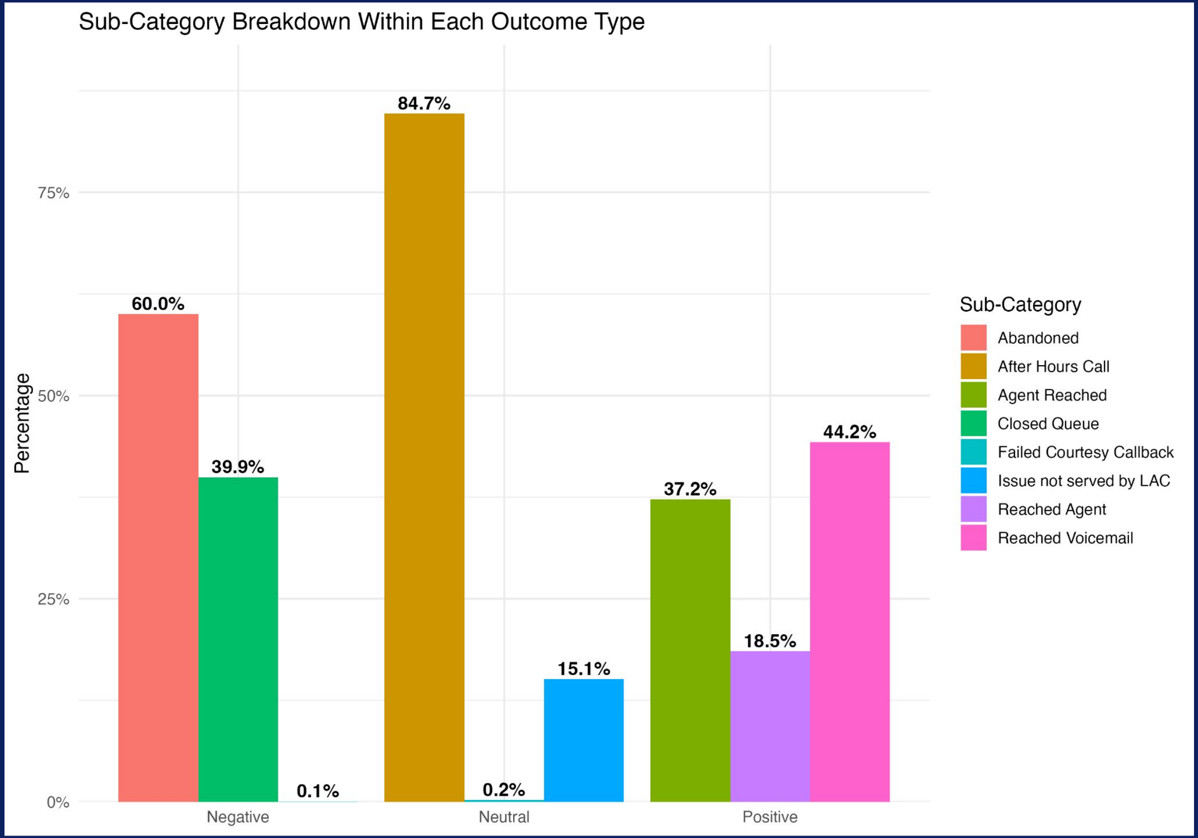


Metric 1: Proportion of positive, negative, and neutral outcomes in the data, and their split into the above sub-categories

Main Takeaways:

Neutral Outcomes:

- After Hours Calls (≈85%) dominate neutral outcomes — most neutral calls happen when Legal Aid is closed.
- Not Served (≈15%) reflects callers whose issue falls outside Legal Aid's scope.

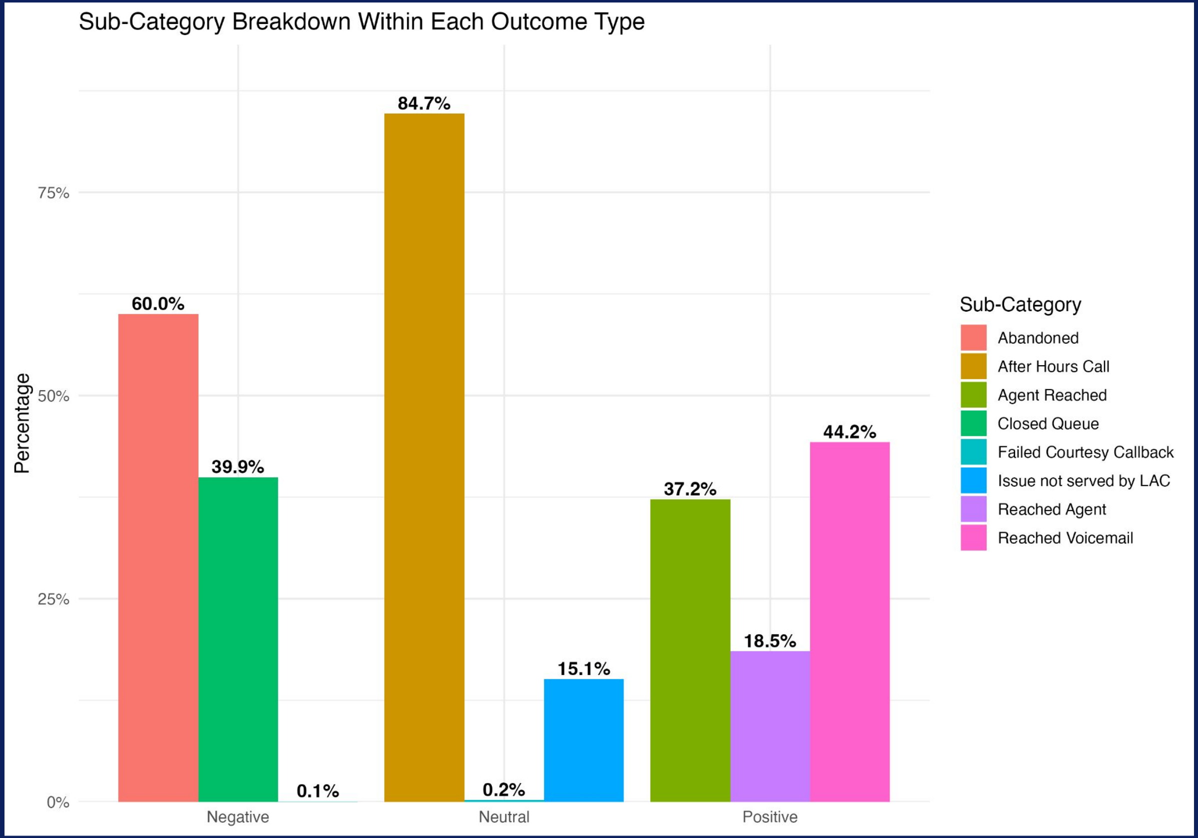


Metric 1: Proportion of positive, negative, and neutral outcomes in the data, and their split into the above sub-categories

Main Takeaways:

Positive Outcomes:

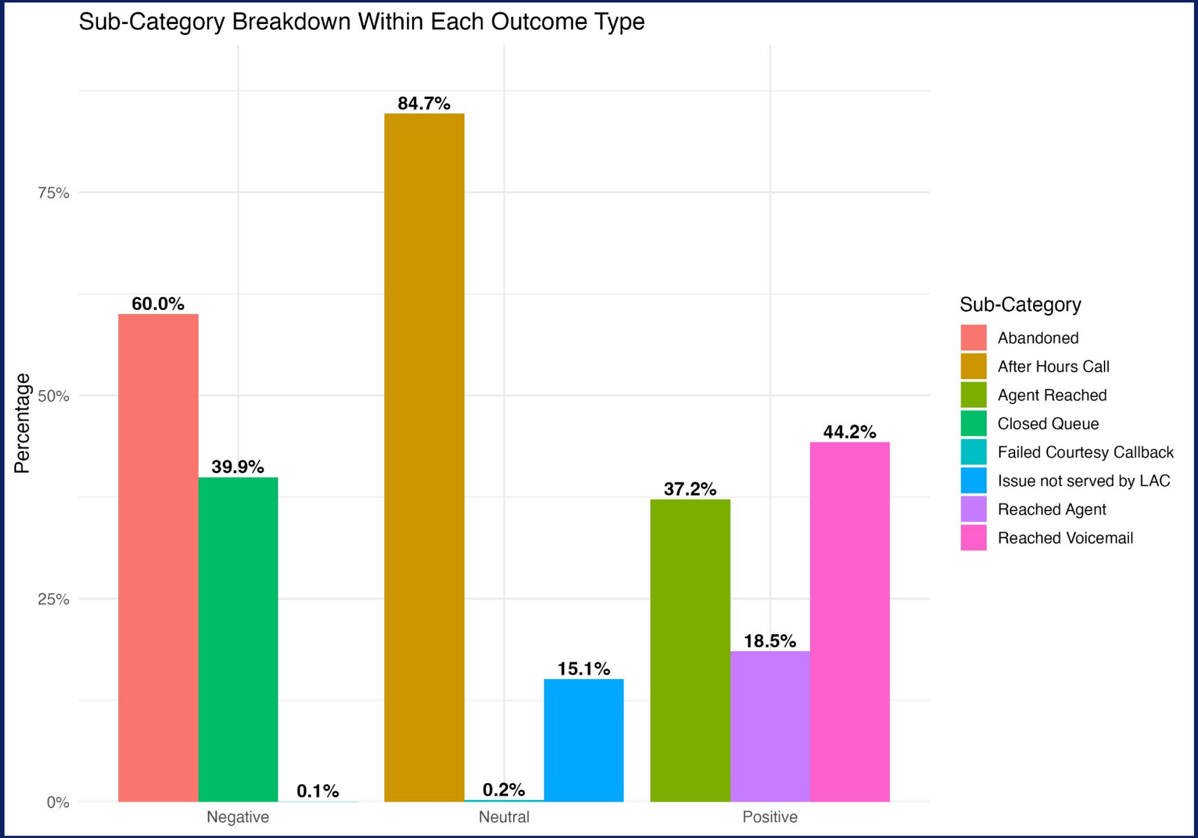
- Positive outcomes are mostly split between Reached Voicemail (≈44%) and Agent Reached (≈37%), showing many callers either get an answer or leave a message.
- A smaller portion (≈18%) comes from Reached Agent via the front desk.



Metric 1: Proportion of positive, negative, and neutral outcomes in the data, and their split into the following sub-categories

Conclusion:

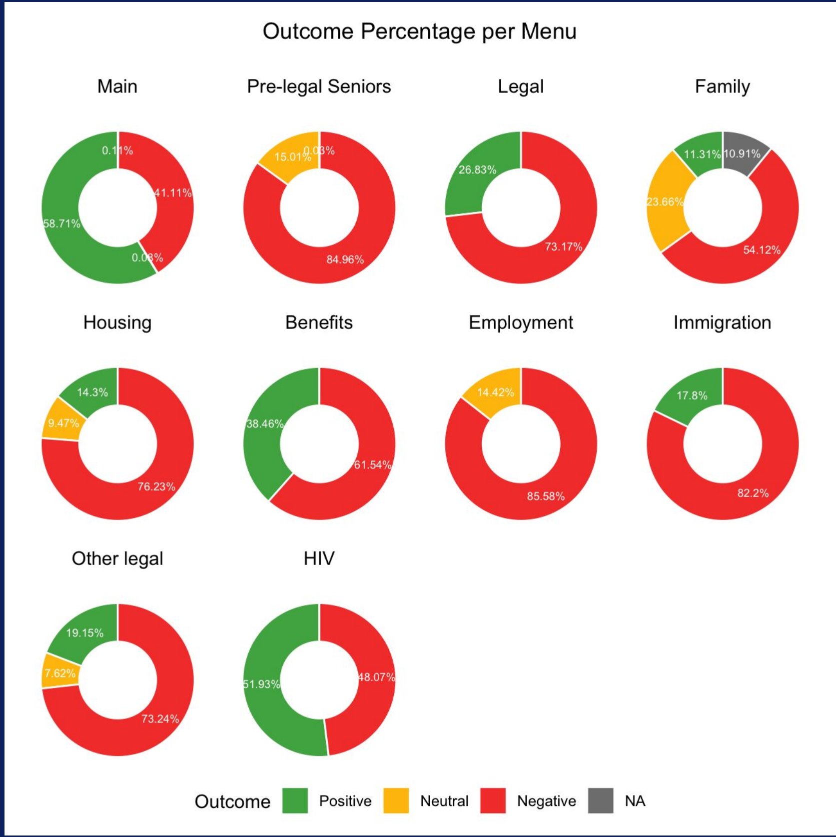
- Most failures occur because callers either cannot reach the system during open hours or abandon the process before connecting.
- Positive experiences tend to happen when callers reach voicemail or an agent, reinforcing the importance of getting callers into a queue that leads to human contact or a message option.



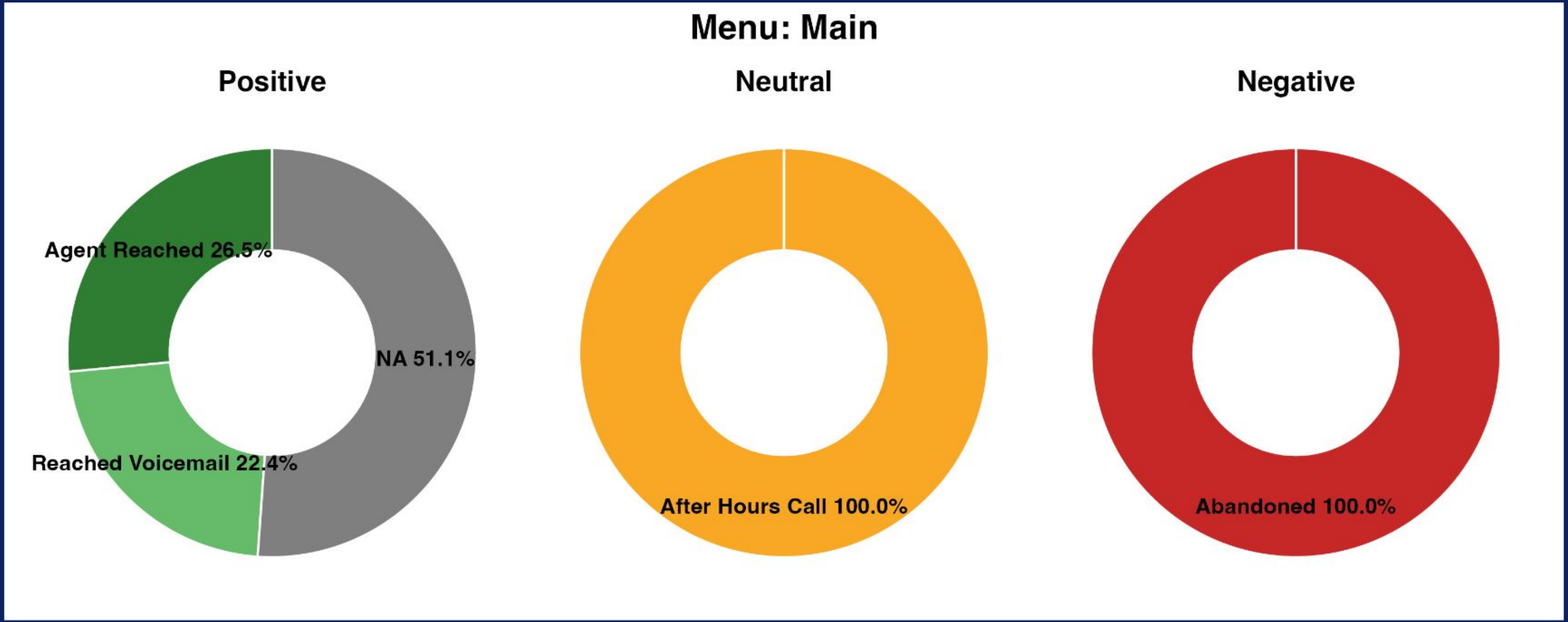
Metric 2.1. Percentages of positive, negative, and neutral outcomes for calls ending in the specific menus

Main takeaways

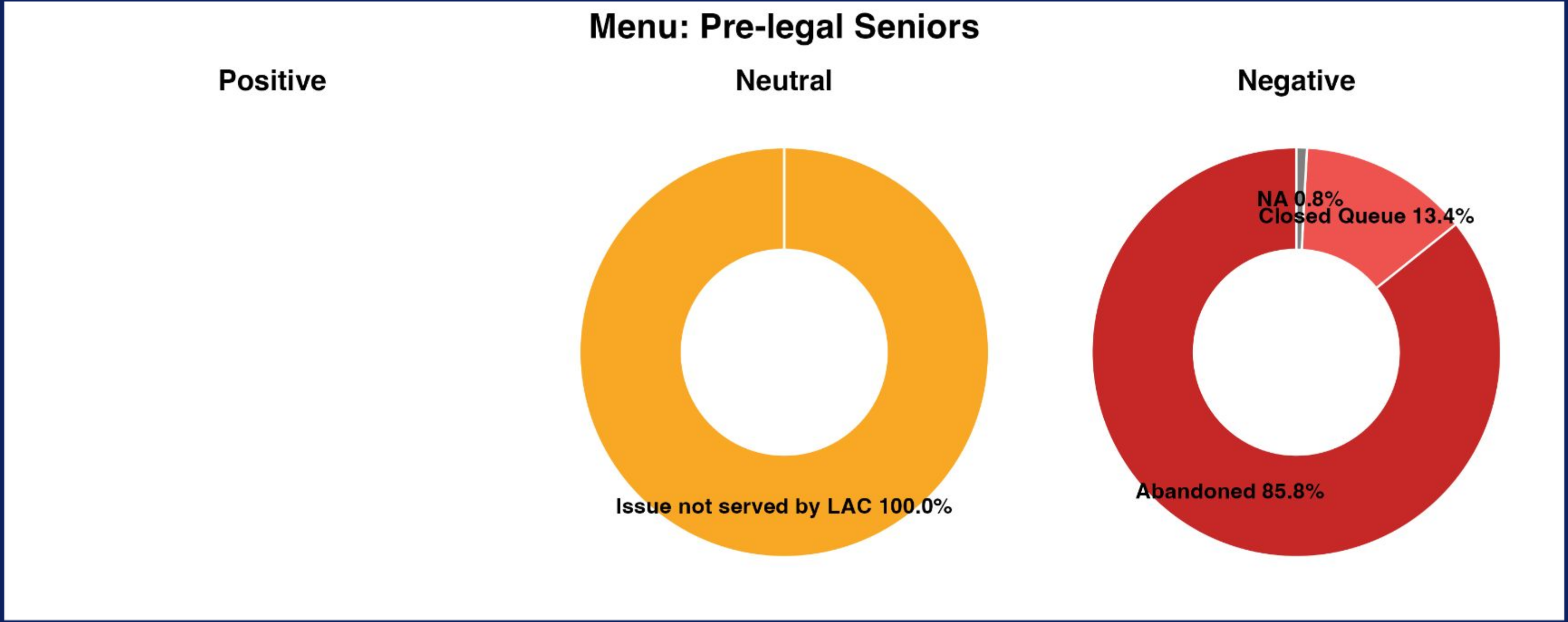
- Most menus have a majority of <u>negative outcomes</u>
  - **Employment** (85.58% negative)
  - **Pre-Legal Seniors** (84.96% negative)
  - **Immigration** (82.2% negative)
  - **Housing** (76.23% negative)
- <u>Positive outcomes</u> vary widely by menu
  - **Main** (58.71% positive)
  - **HIV** (51.93% positive)
  - **Benefits** (38.46% positive)
- **Family** has a noticeable mix of <u>neutral</u> (23.66%) and <u>NA</u> (10.91%) outcomes
  - 51.1% of positive outcomes have not yet been assigned a sub-category
  - All (100%) neutral calls occurred after business hours



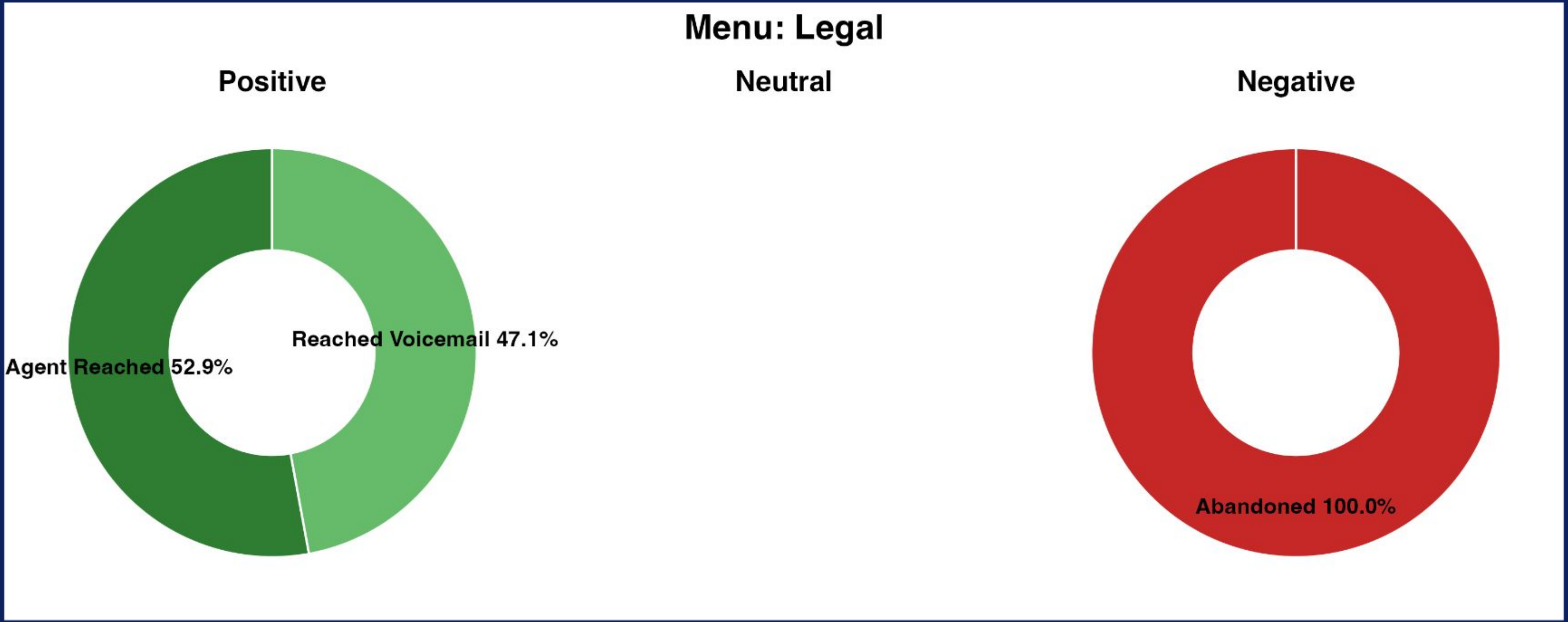
Metric 2.2: By sub-category (Main Menu)



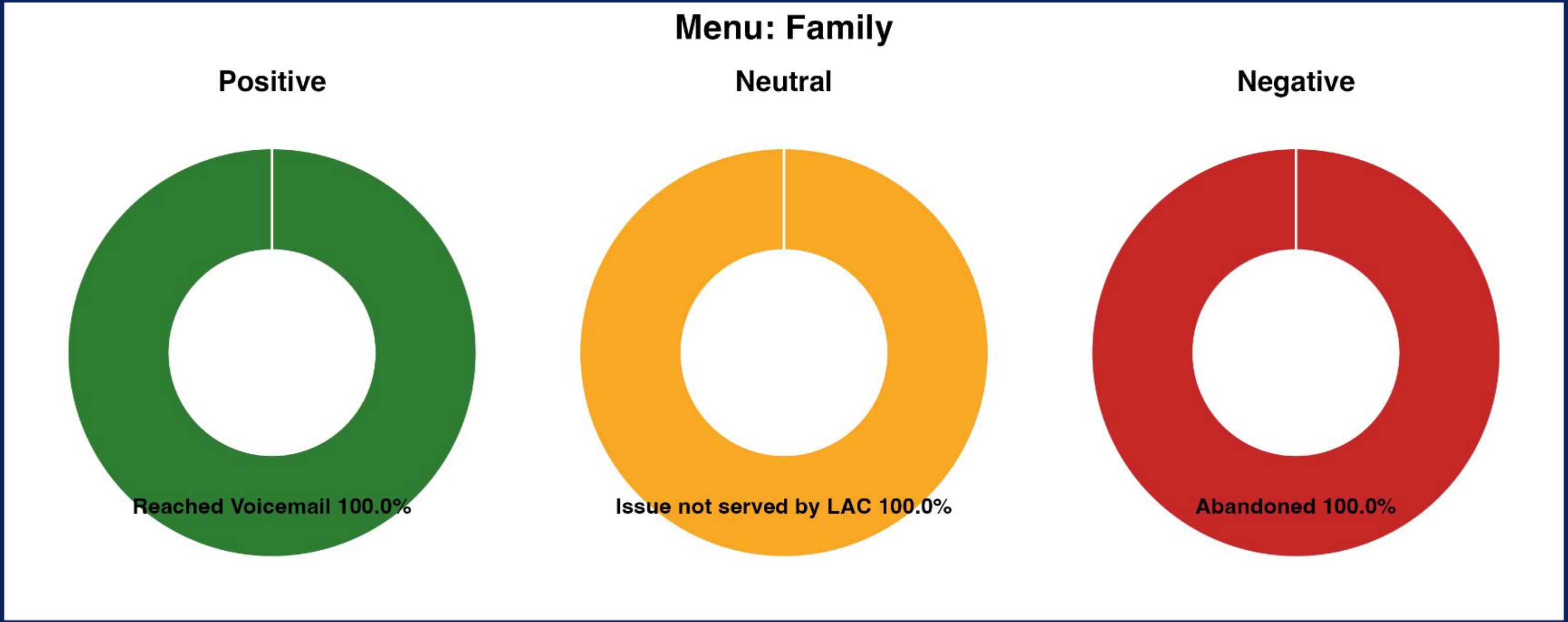
Metric 2.3: By sub-category (Pre-legal Seniors Menu)



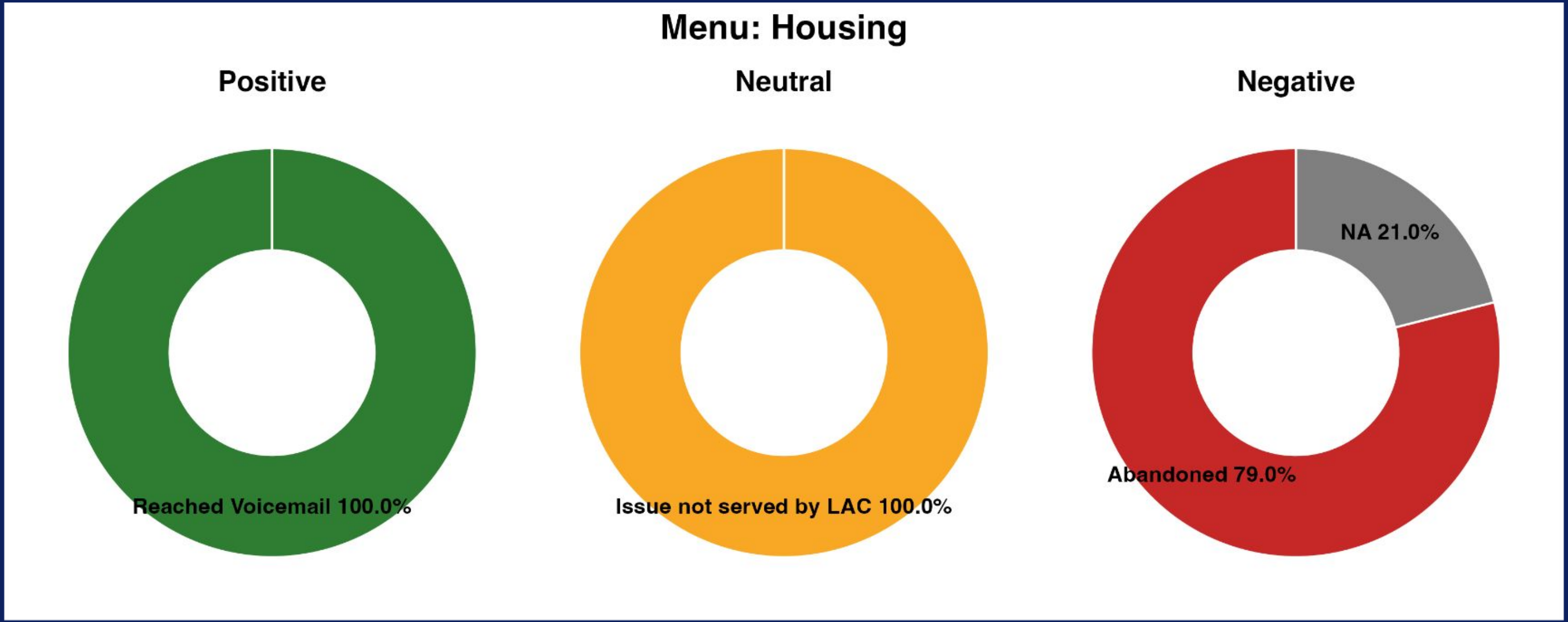
Metric 2.4: By sub-category (Legal Menu)



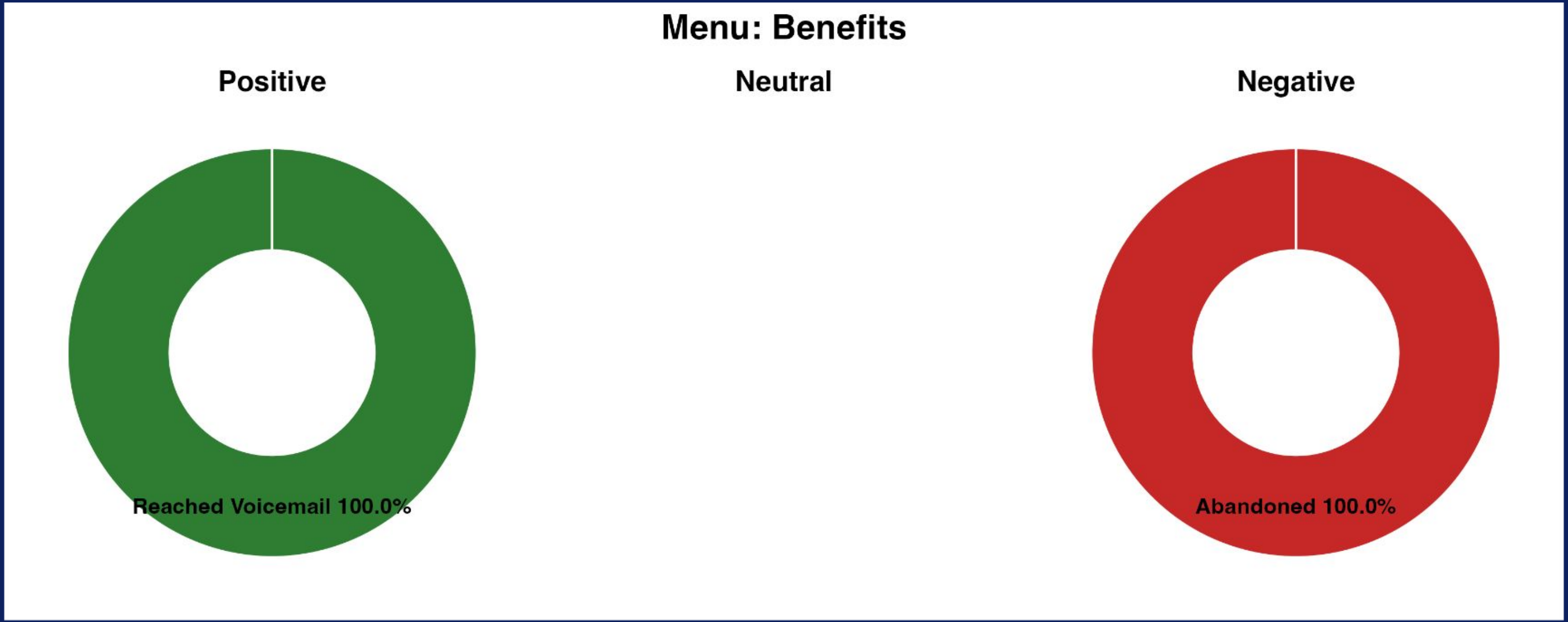
Metric 2.5: By sub-category (Family Menu)



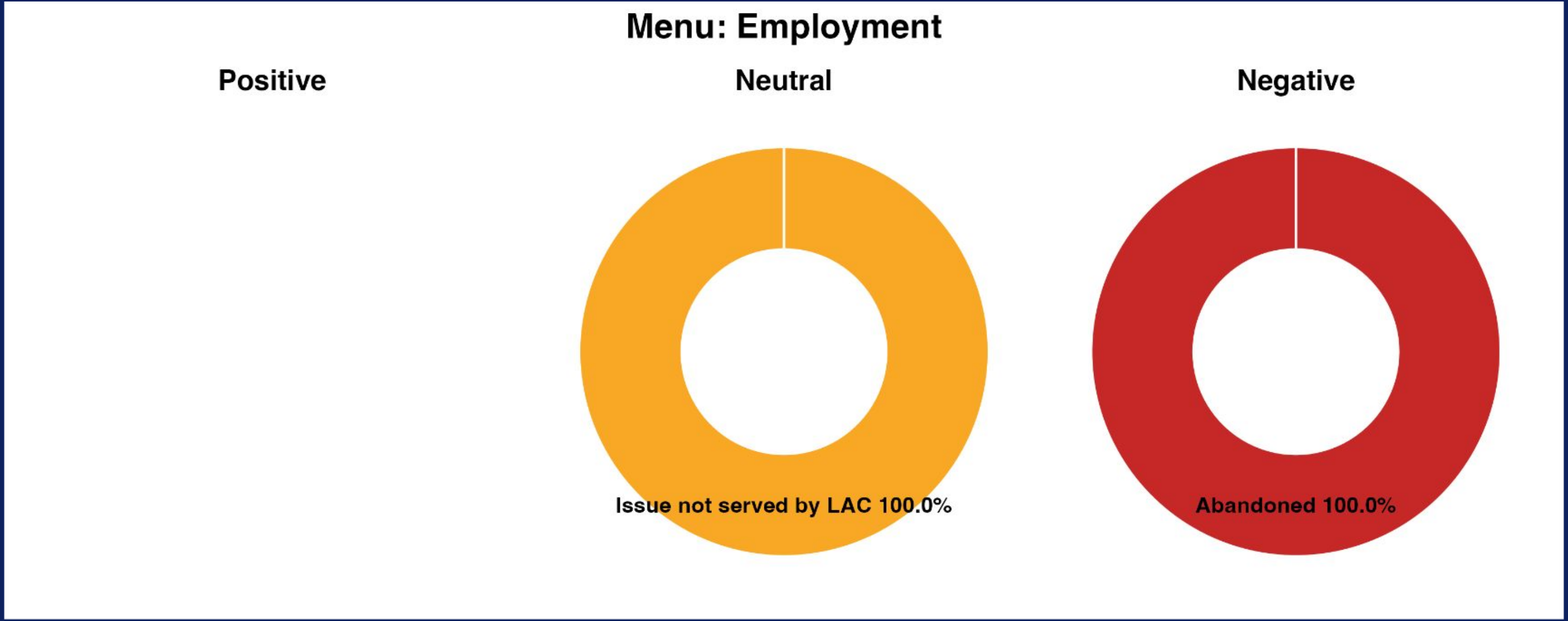
Metric 2.6: By sub-category (Housing Menu)



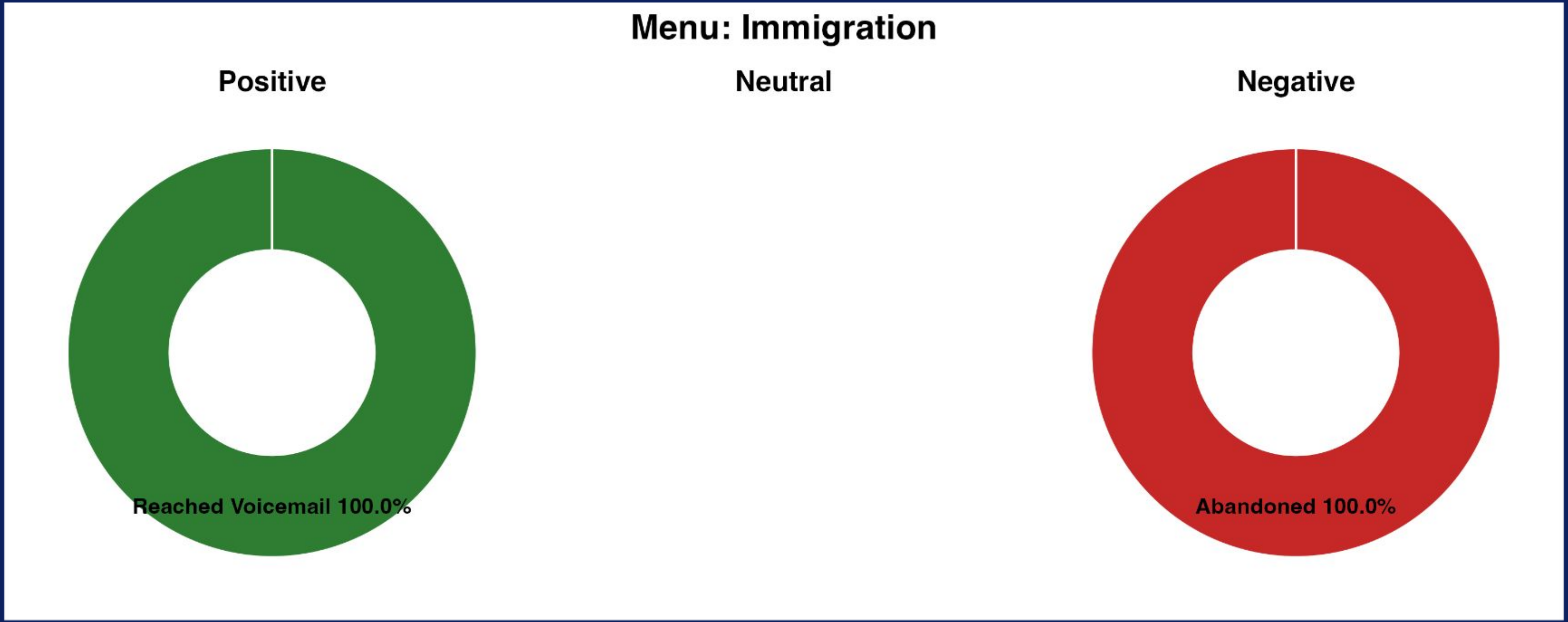
Metric 2.7: By sub-category (Benefits Menu)



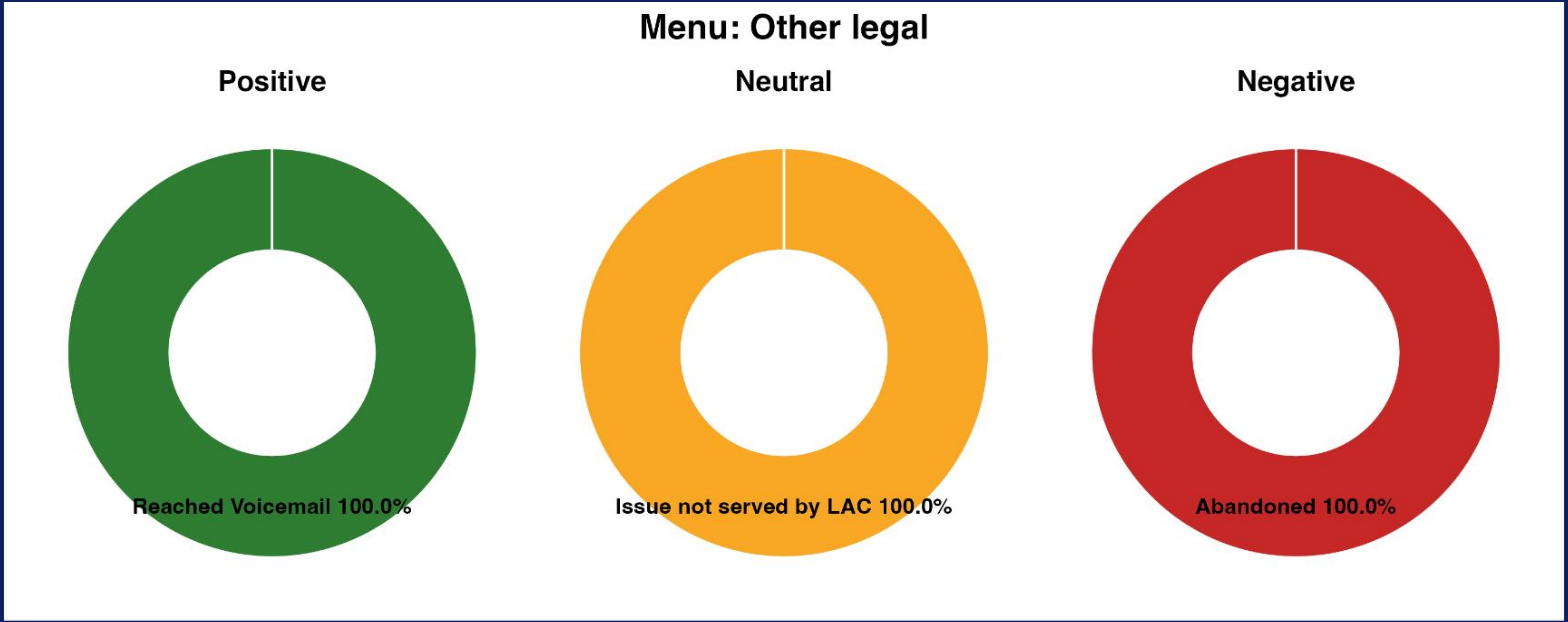
Metric 2.8: By sub-category (Employment Menu)



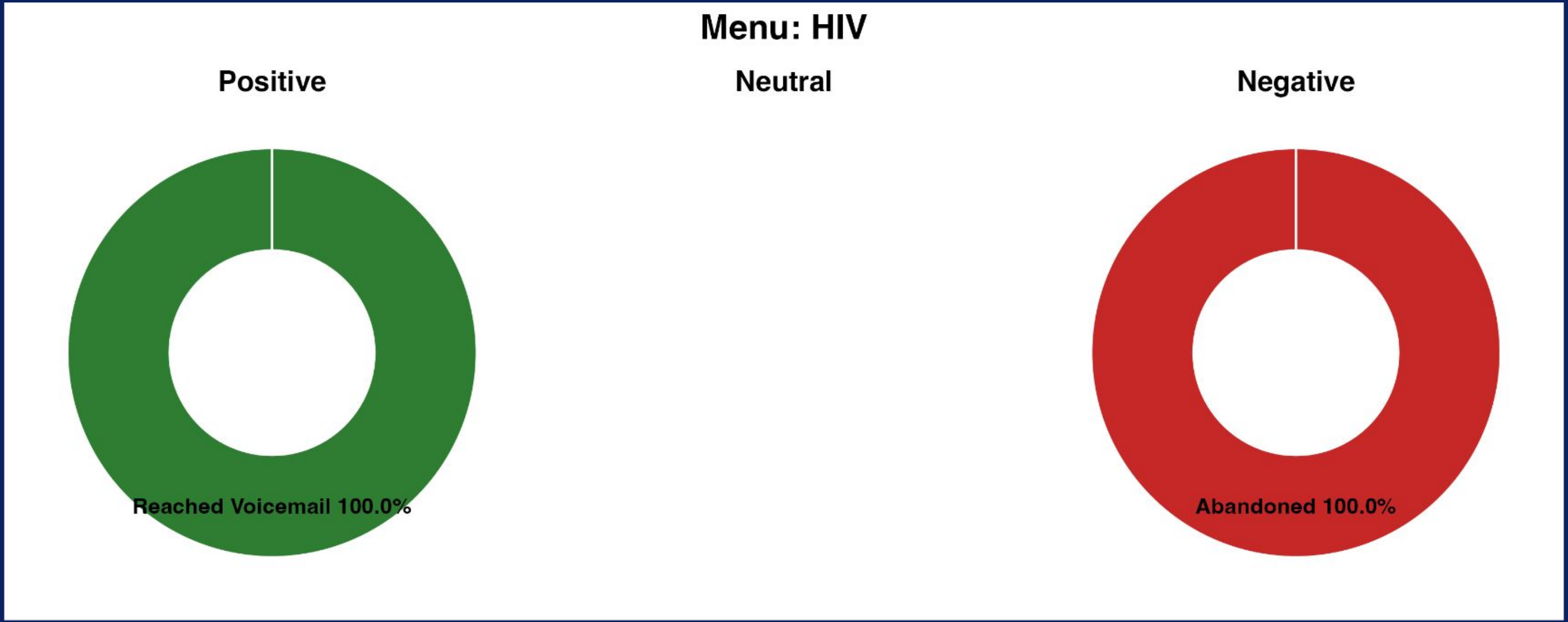
Metric 2.9: By sub-category (Immigration Menu)



Metric 2.10: By sub-category (Other legal Menu)



Metric 2.11: By sub-category (HIV Menu)



Questions & Issues

```
> outcomes_types |>
+   count(last_menu)
# A tibble: 17 × 2
   last_menu                   n
   <chr>                   <int>
 1 ClosedHoursHolidaysMenu 21689
 2 ClosedQueueMenu         49930
 3 CourtesyCallback         9077
 4 FarmworkerMain           9862
 5 Intake_Outdial           9395
 6 LACMain                 98533
 7 LegalBenefitsMenu         481
 8 LegalEmploymentMenu       215
 9 LegalFamilyMenu          3775
10 LegalHIVMenu             1610
11 LegalHousingMenu         5426
12 LegalImmigrationMenu      781
13 LegalMenu               17935
14 OtherLegalMenu          12248
15 PreLegalMenuSeniorsMenu  3957
16 Queues                   3344
17 NA                        184
```

- We identified **low positive outcomes for some menus**
- When analyzing outcomes by menu group, the visualization currently assigns each call to the **last menu the caller accessed** (last_menu)
- However, many calls that ultimately have a positive outcome might flow into:
  - Queue
  - CourtesyCallback
  - Intake_Outdial
- How should we treat other categories besides 10 categories to accurately capture the positive outcomes, rather than being lost when calls move into queues or callback flows?

Thank you!

Appendix

Classifying Outcomes Code & Assumptions

Assumptions

- For final activities that related to being in a queue or courtesy callback, last activity menu was considered to be the menu prior to entering the Courtesy Callback or Queues flow
- For initial classification, considered all FrontDeskTransfer activities as positive; possible next step could be to distinguish between positive and negative outcomes based on time spent in queue or speaking to agent

Next Steps:

- Continuing to finalize classifications and refine rules

Finding Calls with a Target Activity as their Final Activity

```
# Filtering for rows with non-null activity names
df_nonnull = sorted_df[sorted_df["Activity Name"].notna()]

# Findng the last non-null activity names for each contact session ID
last_activities = df_nonnull.groupby("Contact Session ID").tail(1)

#
target = "ScreenPopProcessComplete"
result = last_activities[last_activities["Activity Name"] == target]

session_ids = result["Contact Session ID"].unique()
```

Verifying Outcomes Code

```
for cat in last_act:
    # Find sessions whose LAST activity = this category
    ids = last[last["Last Activity"] == cat]["Contact Session ID"]

    # Extract full journeys for those sessions
    journeys = df_main[df_main["Contact Session ID"].isin(ids)]
    journeys = journeys.groupby('Contact Session ID').tail()

    # Save into dictionary
    results[cat] = {
        "ids": ids,
        "journeys": journeys
    }
```

Grouping Calls by their Last Activity and doing a general lookthrough to verify outcome

Looking at some outcomes more closely

```
# ClinicVoiceMailTransfer
activities = df_main[df_main['Contact Session ID'].isin(last[last['Last Activity'] == "ClinicVoicemailTransfer"]['Contact Session ID'])]
activities.groupby('Contact Session ID').tail()
```

✓ 0.4s

Outcome Tagging Code (R)

```r
car <- read_csv('data/combined_data.csv') |>
       janitor::clean_names() |>
       mutate(activity_start_timestamp = ymd_hms(activity_start_timestamp),
              across(where(is.character), ~ na_if(.x, "N/A")),
              hour = hour(activity_start_timestamp))
```

Read in CAR dataset with few cleaning steps

```r
summary <-
car |>
  summarize(
    .by = contact_session_id,
    last_activity = last(activity_name[!is.na(activity_name)], na_rm = TRUE),
    last_ep =  last(ep_name[!is.na(ep_name)], na_rm = TRUE),
    last_agent = last(agent_name[!is.na(agent_name)], na_rm = TRUE),
    last_menu = last(flow_name[!is.na(flow_name)], na_rm = TRUE),
    last_activity_time =
      last(activity_start_timestamp[!is.na(activity_name)]),
    end_time = last(activity_start_timestamp),
    time_gap = end_time - last_activity_time
  )
```

Create summary table with every contact session ID and final step for each field (non-NA)

Outcome Tagging Code (R)

```r
outcomes_types <-
summary |>
  mutate(
    outcome = case_when(
      # Caroline
      last_activity == 'ClosedQueueMenu' ~ 'Negative',
      last_activity == 'StaffDirectoryEnglishTransfer' ~ 'Positive',
      last_activity == 'LanguageSelectionMenu' ~ 'Negative',
      last_activity == 'ClinicVoicemailTransfer' & last_ep == 'Closed Queue Menu Telephony EP' ~ 'Negative',
      last_activity == 'ClinicVoicemailTransfer' & last_ep != 'Closed Queue Menu Telephony EP' ~ 'Positive',
      last_activity == 'MainMenu' ~ 'Negative',
      last_activity == 'DisconnectContact1' & last_ep == 'Closed Hours-Holidays Menu Telephony EP' ~ 'Neutral',
      last_activity == 'DisconnectContact1' & last_ep != 'Closed Hours-Holidays Menu Telephony EP' ~ 'Negative',
      last_activity == 'LegalMenu2' ~ 'Negative',
      last_activity == 'FarmworkerMainMenu' ~ 'Negative',
```

Tag Pos/Neg/Neutral Outcomes (code is abridged for space purposes))

```r
mutate(
  sub_type = case_when(
    # Caroline
    last_activity == 'ClosedQueueMenu' ~ 'Closed Queue',
    #last_activity == 'StaffDirectoryEnglishTransfer' ~ 'Positive',
    last_activity == 'LanguageSelectionMenu' ~ 'Abandoned',
    last_activity == 'ClinicVoicemailTransfer' & last_ep == 'Closed Queue Menu Telephony EP' ~ 'Closed Queue',
    last_activity == 'ClinicVoicemailTransfer' & last_ep != 'Closed Queue Menu Telephony EP' ~ 'Reached Voicemail',
    last_activity == 'MainMenu' ~ 'Abandoned',
    last_activity == 'DisconnectContact1' & last_ep == 'Closed Hours-Holidays Menu Telephony EP' ~ 'After Hours Call',
    last_activity == 'DisconnectContact1' & last_ep != 'Closed Hours-Holidays Menu Telephony EP' ~ 'Closed Queue',
    last_activity == 'LegalMenu2' ~ 'Abandoned',
    last_activity == 'FarmworkerMainMenu' ~ 'Abandoned',
    last_activity == 'LegalServerScreenPop' & !is.na(last_agent) & time_gap > 0 ~ 'Reached Agent',
```

Tag sub-categories (abridged)

Metric 1 Code (R)

```
metric_1_plot <-
  outcomes_types |>
  filter(!is.na(sub_type), !is.na(outcome)) |>
  count(outcome, sub_type) |>
  group_by(outcome) |>
  mutate(prop = n / sum(n)) |>
  ggplot(aes(x = outcome, y = prop, fill = sub_type)) +

  geom_col(position = position_dodge(width = 0.9)) +

  geom_text(
    aes(label = scales::percent(prop, accuracy = 0.1)),
    position = position_dodge(width = 0.9),
    vjust = -0.3,
    size = 4,
    fontface = "bold"
  ) +

  scale_y_continuous(
    labels = scales::percent,
    expand = expansion(mult = c(0, 0.1))  # extra space above labels
  ) +

  labs(
    title = "Sub-Category Breakdown Within Each Outcome Type",
    x = NULL,
    y = "Percentage",
    fill = "Sub-Category"
  ) +

  theme_minimal(base_size = 12) +
  theme(
    legend.position = "right"
  )
```

1. Prepare and Clean the data
2. Build the bar chart
3. Form the plot

Metric 2.1. Outcome Analysis: Summary table & visualization code

```r
# define the desired order of outcomes for consistent display
outcome_levels <- c("Positive", "Neutral", "Negative")

# prepare the summary table
outcomes_by_menu <- outcomes_types |>
  mutate(
    menu_group = recode(
      last_menu,
      !!!menu_map,
      # ensure unmatched become NA
      .default = NA_character_
    ),
    # enforce outcome order
    outcome = factor(outcome, levels = outcome_levels, ordered = TRUE),
    # enforce menu order
    menu_group = factor(menu_group, levels = unname(menu_map), ordered = TRUE)
  ) |>
  # drop everything not in map
  filter(!is.na(menu_group)) |>
  count(menu_group, outcome) |>
  group_by(menu_group) |>
  mutate(
    percent = round(n / sum(n) * 100, 2)
  ) |>
  ungroup() |>
  arrange(menu_group, outcome)
```

```r
# visualization 2.1 from the presentation
ggplot(outcomes_by_menu, aes(x = 2, y = percent, fill = outcome)) +
  geom_col(width = 1, color = "white") +
  geom_text(aes(label = paste0(percent, "%")),
            position = position_stack(vjust = 0.5),
            color = "white",
            size = 2.3,
            # check_overlap = TRUE
  ) +
  coord_polar(theta = "y") +
  xlim(0.5, 2.5) +
  facet_wrap(~menu_group) +
  scale_fill_manual(
    values = c("Positive" = "#4CAF50",
               "Neutral"  = "#FFC107",
               "Negative" = "#F44336")
  ) +
  theme_void() +
  theme(
    strip.text = element_text(size = 11, margin = margin(b = 10)),
    legend.position = "bottom",
    plot.title = element_text(hjust = 0.5, margin = margin(b = 20)),
    plot.margin = margin(t = 20, r = 10, b = 10, l = 10)
  ) +
  labs(fill = "Outcome", title = "Outcome Percentage per Menu")
```

Metric 2.2.-2.11. Sub-outcome Analysis: Summary table code & result

```r
## sub-outcome analysis ----
sub_outcomes_by_menu <- outcomes_types |>
  mutate(
    menu_group = recode(
      last_menu,
      !!!menu_map,
      .default = NA_character_
    ),
    outcome = factor(outcome, levels = outcome_levels, ordered = TRUE),
    menu_group = factor(menu_group, levels = unname(menu_map), ordered = TRUE)
  ) |>
  filter(!is.na(menu_group)) |>
  count(menu_group, outcome, sub_type) |>
  group_by(menu_group, outcome) |>
  mutate(
    percent = round(n / sum(n) * 100, 2)
  ) |>
  ungroup() |>
  arrange(menu_group, outcome, sub_type)

# custom sub-type palettes (3 shades each)
pal_positive <- c("#2E7D32", "#66BB6A", "#A5D6A7")
pal_neutral  <- c("#F9A825", "#FFCA28", "#FFE082")
pal_negative <- c("#C62828", "#EF5350", "#FFCDD2")

menus <- unique(sub_outcomes_by_menu$menu_group)
```

menu_group	outcome	sub_type	n	percent
Main	Positive	Agent Reached	15326	26.49
Main	Positive	Reached Voicemail	12950	22.39
Main	Positive	NA	29571	51.12
Main	Neutral	After Hours Call	74	100.00
Main	Negative	Abandoned	40504	100.00
Main	NA	NA	108	100.00
Pre-legal Seniors	Neutral	Issue not served by LAC	594	100.00
Pre-legal Seniors	Negative	Abandoned	2883	85.75
Pre-legal Seniors	Negative	Closed Queue	451	13.41
Pre-legal Seniors	Negative	NA	28	0.83
Pre-legal Seniors	NA	NA	1	100.00
Legal	Positive	Agent Reached	2545	52.89
Legal	Positive	Reached Voicemail	2267	47.11
Legal	Negative	Abandoned	13123	100.00
Family	Positive	Reached Voicemail	427	100.00
Family	Neutral	Issue not served by LAC	893	100.00
Family	Negative	Abandoned	2043	100.00
Family	NA	NA	412	100.00
Housing	Positive	Reached Voicemail	776	100.00
Housing	Neutral	Issue not served by LAC	514	100.00
Housing	Negative	Abandoned	3266	78.97
Housing	Negative	NA	870	21.03
Benefits	Positive	Reached Voicemail	185	100.00
Benefits	Negative	Abandoned	296	100.00
Employment	Neutral	Issue not served by LAC	31	100.00
Employment	Negative	Abandoned	184	100.00
Immigration	Positive	Reached Voicemail	139	100.00
Immigration	Negative	Abandoned	642	100.00
Other legal	Positive	Reached Voicemail	2345	100.00
Other legal	Neutral	Issue not served by LAC	933	100.00
Other legal	Negative	Abandoned	8970	100.00
HIV	Positive	Reached Voicemail	836	100.00
HIV	Negative	Abandoned	774	100.00

Metric 2.2.-2.11. Sub-outcome Analysis: Visualization code

```r
# visualization 2.2-2.11 from the presentation
for (m in menus) {

  df_menu <- sub_outcomes_by_menu |>
    filter(menu_group == m)

  # function to build one donut per outcome
  build_donut <- function(outcome_name, palette) {
    df_out <- df_menu |> filter(outcome == outcome_name)

    ggplot(df_out, aes(x = 2, y = percent, fill = sub_type)) +
      geom_col(width = 1, color = "white") +
      scale_fill_manual(values = palette) +
      coord_polar(theta = "y") +
      xlim(0.5, 2.5) +
      geom_text(
        aes(
          label = paste0(sub_type, " ", sprintf("%.1f%%", percent))
        ),
        position = position_stack(vjust = 0.5),
        size = 3.6,
        fontface = "bold",
        check_overlap = TRUE
      ) +
      coord_polar(theta = "y", clip = "off") +
      theme_void() +
      theme(
        legend.position = "none",
        plot.title = element_text(hjust = 0.5, face = "bold")
      ) +
      ggtitle(outcome_name)
  }
```

```r
# build the 3 donuts with distinct palettes
p_pos <- build_donut("Positive", pal_positive)
p_neu <- build_donut("Neutral",  pal_neutral)
p_neg <- build_donut("Negative", pal_negative)
# combine horizontally
row_plot <- p_pos + p_neu + p_neg + plot_layout(ncol = 3)
# add title
final_plot <- (
  row_plot +
    plot_annotation(
      title = paste("Menu:", m),
      theme = theme(
        plot.title = element_text(size = 16, face = "bold", hjust = 0.5)
      )
    )
)
```